Phase 1: Assisted Development

Module 3: Dynamic Context & Feedback Loops

Establishing a collaboration contract

👋 Agenda

- Learning Objectives
- Environment details and dev scripts
- Workflow Process and our status
- Component of your personalised workflow
    - Scope: 2-stages of planning: story & task
    - Work item file: acceptance criteria, status
    - Proposal: Plan, build/assess, reflect/adjust, commit/pick_next

🚶 Journey Checkpoint

- **Module 1**: Collaboration foundations
  - Wrote code and tests with your assistant
  - Learning to communicate intent, not just instructions
- **Module 2**: The WHAT — semantic context
  - ABOUT, ARCHITECTURE, IMPLEMENTATION, AGENTS.md
  - Your assistant knows what you're building
- **Module 3**: The HOW — procedural and episodic context
  - ENV_SCRIPTS, WORKFLOW_STATUS, work items
  - Your assistant knows how you build, and where you are

📚 Learning Objectives

- **Goal**: Design and test a collaborative workflow
- **Takeaway**: A workflow process you can use/improve going forward
- **Hope**: You notice flow is easier to find and stay in

Along the way you will:

1. **Collaboratively** craft a process that prioritises quality, learning, and flow
2. **Take control** when you notice your assistant loosing their way
3. **Mitigate friction** by investing in our process

📜 Developer Scripts

- `memory/ENV_SCRIPTS.md`
- Automated testing: `make test`
- Static analysis: `make lint` and `make format`
- Builds and local deploys: `make build` and `make up`
- Documented so your assistant can use them

🧪 Dynamic Context

Static context is what you provide upfront — your memory files, AGENTS.md.

Dynamic context is what the agent goes and gets:

- File contents it reads on demand
- Test output, console logs, build results
- Web research, API responses

This enables a **feedback loop**: static context shapes how the agent reasons; the agent acts; the results become new context that informs the next step.

⚙️ Workflow Process

- `memory/WORKFLOW_STATUS.md`
  - Maintains high-level status
  - Defines the stages of the process
- `changes/00x-name_of_story_work_item.md`
  - Contains the status and specific details of the work

✌️ Two Stages of Planning

- Story planning:
  - Prerequisite to iterate on a work item
- Task planning:
  - Performed iteratively
  - Constructed collaboratively

📖 Work item (story/feature) File

- Captures ALL planning discussions/decisions
- Knows what implementation stage we're in
- Maintains a list of all tasks for completion
- Contains all planning for current task
    - Pruned at the end of the task
- The right amount of history in the repo

🗓️ Stages

> A starting point/proposal

1. Task Planning*
2. Build* & Assess
3. Reflect* & Adapt
4. Commit & Pick next

All stages are worked on collaboratively (* are required)

The more disciplined your process, the more empowered your assistant, and the better results you'll get. **AI tools reward intentionality.**

🗓️ Stage 1: Task Planning

- *Can be* the most time consuming stage
- Where you get to control the scope of work (overwhelm)
- Feasibility, scope, patterns, dependencies, examples
- Inputs: from the backlog, voice note, etc + existing code
- Outputs: verifiable acceptance criteria in story file

🛠️ Stage 2: Build & Assess

- Where you get to partition the work
- Write tests, run them, assert failures
- Implement a test, run it, assert success
- Refactor, lint, type check, run tests
- Review code for pattern adherence, complexity, correctness, etc
- Your assistant in a feedback loop, with you guiding it
- Use the Git working tree to inform status
- Incrementally applying a pattern across the codebase (too large to apply at once)?
  — Ensure you applying required changes

🤔 Stage 3: Reflect & Adapt

On the process — not the implementation (that's Stage 2):

- Did PLAN and BUILD go as expected? What would reduce friction?
- How can you better express your intentions next time?

On the knowledge graph:

- Update context files that changed meaning during this work
- Check READMEs in folders you touched — still accurate?
- Purge stage scaffolding from the work item; keep outcomes

> A reflection stage (agent retro) is highly recommended.

📦 Stage 4: Commit & Pick next

- Verify acceptance criteria are met
- Update `WORKFLOW_STATUS.md` — current item done, next item named
- Review READMEs in folders you touched; update any that drifted
- Purge PLAN/BUILD/REFLECT docs from the work item file; keep criteria and outcomes
- Commit with intention; start the next unit of work

🚀 Let's have a play

1. Start with PLAN (and don't be in a hurry)
2. If BUILD goes horribly wrong, start over
3. Maintain awareness of the *process*